
Oficina Mecânica API

Gestão de ordens de serviço para uma oficina de médio porte.
Da regra de negócio ao cluster, com a esteira inteira no meio.

Danilo Canato — RM 372411 — Discord: @canatodanilo

Guilherme Fumagali Marques — RM 372421 — Discord: @gfumafali

Tech Challenge · Fase 2 · PosTech FIAP — Arquitetura de Software

O que já existe

A Fase 1 entregou o MVP. Esta fase parte dele — inclusive da dívida que deixamos anotada.

O PRODUTO

MVP em Clean Architecture

Back-end monolítico containerizado.
Domínio em Java puro, sem framework. O ciclo de vida completo da OS: 8 endpoints, snapshot de preço, estorno atômico de estoque.

O MODELO

DDD, estratégico e tático

Event Storming e Context Map. 7 bounded contexts. OrdemServico como único Aggregate Root. JWT + refresh token com rotação e detecção de reúso.

O RIGOR

104 testes, gate bloqueante

JaCoCo com enforce de 80% no domínio. OWASP Dependency-Check e SonarCloud barrando a main.

- **E três débitos técnicos:** mappers manuais, um construtor de nove parâmetros, e o serviço de notificação sem canal concreto — o Hot Spot PA2 do Event Storming, deixado em aberto porque isso ainda não tinha sido decidido.

O que esta entrega atende

Novas regras de negócio e a cadeia de infraestrutura completa, do container ao cluster.

DOMÍNIO

Três regras novas

A fila de OS ordenada por prioridade de negócio, não por data. Exclusão lógica de Finalizada e Entregue. E o cliente aprovando o orçamento por token, sem login, pelo e-mail que recebe.

INFRAESTRUTURA

Do container ao cluster

Imagem enxuta, Kubernetes com escalabilidade automática, Terraform provisionando do zero e uma esteira que vai do commit ao deploy.

DÍVIDA TÉCNICA

Os três débitos, quitados

Mappers manuais trocados pela lib MapStruct, que gera o mapeamento no build. O construtor de nove parâmetros virou um record. E a notificação ganhou canal: SMTP real — fechando o Hot Spot que a Fase 1 deixou aberto.

Build e produção, separados por construção

Build multi-stage: a etapa que constrói é descartada, e a imagem final carrega só o necessário para executar.

```
# etapa 1 – compila
FROM maven:3.9-eclipse-temurin-21 AS builder
RUN mvn package -DskipTests

# etapa 2 – o que realmente vai pra produção
FROM eclipse-temurin:21-jre-alpine
RUN adduser -S spring
COPY --from=builder /app/target/*.jar app.jar
USER spring
HEALTHCHECK CMD wget -qO- /actuator/health
```

- **Sem Maven, sem fonte, sem cache.** A imagem final tem só o JRE e o JAR.
- **Usuário não-root.** E um healthcheck declarado no próprio container.
- **Serviços separados.** docker compose up sobe API, Postgres e MailHog.

O que faz o autoscaling funcionar

Não é o HPA sozinho. São três peças, e faltar uma delas quebra tudo em silêncio.

RÉPLICAS

De 2 a 8, pela carga

O HPA acompanha CPU e memória e sobe réplicas quando a carga aperta — com atraso proposital, para não ficar oscilando a cada pico.

A PEÇA QUE TODO MUNDO ESQUECE

Sem requests, não há conta

O HPA calcula uso sobre o que o pod pediu. Sem requests declarados, ele não tem denominador — e simplesmente não escala.

PROBES

Três, cada uma com um papel

A startup dá 5 min para o Spring subir e segura as outras. A readiness controla o tráfego. A liveness reinicia o pod travado.

- **Sem startupProbe, a liveness mata o app durante o boot.** Ela conta desde o início e derruba um pod saudável que só estava demorando a subir. O sintoma é CrashLoopBackOff — e parece bug da aplicação.
- **O metrics-server é responsabilidade nossa.** Nem o kind nem o EKS o trazem de fábrica. Sem ele, o HPA lê <unknown> para sempre e nunca escala.

O que o Terraform provisiona na AWS

19 recursos em us-east-1: rede, IAM, EKS e RDS — do CIDR da VPC à classe da instância do banco.

```
resource "aws_eks_node_group" "oficina" {
  instance_types = ["t3.small"]
  scaling_config {
    desired_size = 2
    min_size     = 2
    max_size     = 3
  }
}

resource "aws_db_instance" "oficina" {
  engine           = "postgres"
  engine_version   = "16"
  instance_class    = "db.t4g.micro"
  allocated_storage = 20
  storage_type      = "gp3"
  multi_az         = false
}
```

- **Rede.** VPC 10.0.0.0/16, duas subnets públicas (us-east-1a e 1b), Internet Gateway e rota default. Sem NAT Gateway: os nós ficam em subnet pública, com Security Group restritivo — trade-off aceito porque o cluster é efêmero.
- **IAM.** Duas roles — cluster e nós — com as quatro managed policies que o EKS exige.
- **Computação.** EKS 1.33, node group gerenciado de 2 a 3 nós t3.small.
- **Exposição.** Service type LoadBalancer: o cloud controller do EKS provisiona um ELB e devolve o hostname público.

O backend de state do Terraform

O Terraform registra num arquivo — o state — tudo que já provisionou. É por ele que a ferramenta sabe o que existe e o que falta criar. Guardamos esse arquivo num bucket S3, com trava no DynamoDB. Mas quem cria o bucket?

COMO É FEITO

S3 versionado, trava no DynamoDB

O state vive num bucket S3 versionado e criptografado. A trava no DynamoDB impede que dois apply simultâneos corrompam o registro.

O IMPASSE

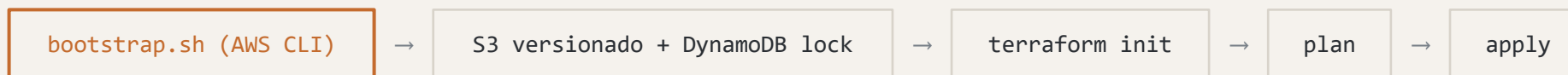
Ele não cria o próprio backend

O primeiro apply — o que criaria o bucket — não teria onde guardar o próprio state. Em disco não serve: o runner do CI é efêmero e leva o arquivo embora.

A SAÍDA

Um script sem estado

Bucket e tabela são criados por um script de AWS CLI idempotente. Sem state para perder, ele converge sempre no mesmo lugar — e roda dez vezes sem susto.



Os cinco passos são jobs do mesmo workflow: o bootstrap é o job 1, e roda antes do plan.

Do commit ao cluster

Duas esteiras no mesmo repositório: uma entrega a aplicação, a outra a infraestrutura.

APLICAÇÃO



INFRAESTRUTURA



AUTENTICAÇÃO

OIDC, sem chave estática

O GitHub troca um token de curta duração por uma role que só este repositório consegue assumir. Nenhuma credencial de longa duração para vazar ou rotacionar.

CONTROLE DE MUDANÇA

O plan é revisado antes do apply

O plan vira artefato e a esteira para num Environment com required reviewers. Separa quem propõe de quem aplica — e o apply usa exatamente o plano revisado.

RASTREABILIDADE

A imagem carrega o SHA do commit

A tag da imagem é o hash do commit. O que roda no cluster é rastreável até a linha de código que o gerou.

O que entregamos

Da regra de negócio ao cluster em nuvem — e os conceitos das aulas que cada etapa exercita.

APLICAÇÃO

A arquitetura sustentou a evolução

As três regras novas entraram sem tocar em infraestrutura. É o teste que a Clean Architecture precisa passar — e passou.

INFRAESTRUTURA

Do container à nuvem

Imagem multi-stage, Kubernetes com probes e escalabilidade automática, e Terraform provisionando VPC, EKS e RDS do zero.

AUTOMAÇÃO

A esteira liga as duas pontas

Do commit ao cluster: quality gates a cada push, autenticação federada por OIDC e controle de mudança na infraestrutura.

- **Conceitos exercitados:** Clean Architecture e DDD · containerização · orquestração e escalabilidade · infraestrutura como código · CI/CD e cultura DevOps.

A aplicação, rodando no EKS

- 1 Login → JWT. Access de 15 minutos, refresh de 7 dias com rotação e detecção de reuso.
- 2 Cadastrar cliente (CPF com dígito verificador real) e veículo (placa Mercosul).
- 3 Abrir a OS e iniciar o diagnóstico.
- 4 Adicionar serviço (snapshot de preço) e peça — e ver o estoque cair na hora.
- 5 Gerar o orçamento. A API dispara o e-mail para o cliente, com o token de aprovação.
- 6 Aprovar internamente, com JWT: o caminho do funcionário. Concluir e entregar.
- 7 Numa OS nova: o cliente aprova pelo token do e-mail, no endpoint público, sem login.

base = ELB público do cluster EKS · roteiro completo em docs/demo.http

Os mesmos manifestos, dois clusters — e o HPA reagindo à carga

- 1 kind: um terraform apply cria o cluster, constrói a imagem e sobe tudo. Custo zero.
- 2 AWS: o mesmo Terraform provisiona VPC, EKS e RDS — e o ELB que a demo anterior usou.
- 3 kubectl get pods nos dois clusters: os manifestos são exatamente os mesmos.
- 4 Tela dividida: o HPA de um lado, o k6 gerando carga do outro.
- 5 Subir a carga e acompanhar o HPA — de <unknown> ao percentual real, passando de 70%.
- 6 As réplicas sobem: 2, 3, 4. Quatro é o teto que definimos no manifesto.

Obrigado

- **Regras novas sem tocar em infraestrutura.** A arquitetura aguentou.
- **Infraestrutura declarada e versionada.** E destruível com um clique.
- **A esteira inteira.** Do commit ao cluster.

Danilo Canato — RM 372411 — Discord: @canatodanilo

Guilherme Fumagali Marques — RM 372421 — Discord: @gfumafali